PASS Data Community Summit
Host redgate

Would you notice? Governing AI assistants and MCP servers on your databases

Kellyn Gorman
She/Her
Engineer & Advocate
Redgate

John Martin
He/Him
Global Partner Solution Architect
Redgate

Kellyn Gorman

She/Her

Engineer & Advocate
Redgate



I work on how teams run databases across platforms and clouds, with a focus on DevOps, observability and, lately, putting guardrails around AI in database work.

I blog at dbakevlar.com.

Kellyn Gorman

John
Martin

He/Him



Global Partner Solutions Architect
Redgate

I spend my time working with people to help them get the most out of their data. Helping to bring people, processes, and technology together to solve problems and help them succeed.

/in/johnqmartin

Agenda

01 

The risks, and why identity comes first

Assistants, MCP servers and the one thing the database can see

02 

A practical scenario

Where access quietly grows, one reasonable step at a time

03 

Spotting it in the workload you already monitor

Signals, queries and a Redgate Monitor demo

04 

A hardened baseline that shows drift

CIS Benchmarks and configuration compliance, with a demo

05 

“What did the AI assistant actually do?”

A practical starting point for answering it

AI is already part of database work

Governance hasn't kept up

44%

of organizations use AI in database management

up from 15% a year earlier

23%

use a formal data governance or quality framework

the rest are relying on habit

Source: Redgate, State of the Database Landscape 2026

PASS Data Community Summit

DAMA - DAMBOK

- Standardization
- Governance & Compliance
- Strategic Alignment
- Operational Efficiencies

AI is already part of database work

Governance hasn't kept up

44%

of organizations use AI in database management

up from 15% a year earlier

23%

use a formal data governance or quality framework

the rest are relying on habit

58%

accept higher risk as a trade-off for efficiency

speed is winning the argument

Source: Redgate, State of the Database Landscape 2026

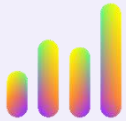
PASS Data Community Summit

It's already connected



A developer chasing a bug

Adds a SQL Server MCP server to their IDE assistant and points it at the database where the bug lives.



An analyst querying data

Asks a chat assistant questions in plain English. It writes and runs the SQL against a reporting copy, or production.



Agents and automation

Scheduled or event-driven agents with tool access, running without a person watching each step.

The question isn't whether to allow it. It's whether you'd notice if something went wrong.

01

The risks

Assistants, MCP servers, and why identity comes first

PASS
Data Community Summit

How an assistant reaches your database



Person

Asks in plain English



AI assistant

Model plans the steps



MCP server

Exposes tools



Database

Sees one login

Risk "Just fix it" is a valid prompt

Risk Can be steered by text it reads

Risk run_query, list_tables, maybe execute

Risk Its rights are the model's rights

The database only sees the last hop. Identity is the only thing that tells you who is really there.

Six risks to plan for

Over-privileged credentials

Whatever login the MCP server uses, the model inherits, including db_owner and sysadmin.

Shared identity

A shared or personal login makes the assistant invisible in every log you have.

Prompt injection through data

Text in a row, a ticket or a document can steer the next tool call.

Unbounded workload

Machine-speed queries, schema crawls and full scans on production, in business hours.

Data leaving the boundary

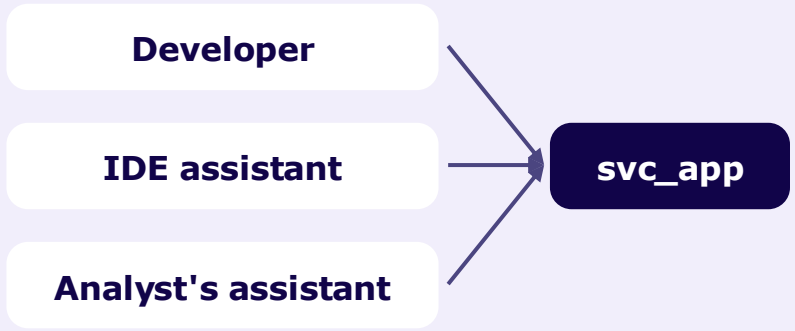
Result sets go to a model provider and live on in chat history and logs.

Write and DDL tools

One “fix it” prompt away from an UPDATE without a WHERE clause.

Every entity needs its own identity

Shared identity



What the log says

```
svc_app   SELECT * FROM dbo.Customers
```

Who ran it? No way to tell.

One identity per entity

Developer	**CONTOSO\kgorman**	their own work
IDE assistant	**mcp_dev_ro**	read-only, staging
Analyst's assistant	**mcp_reporting_ro**	readable secondary
Triage agent	**agent_triage_ro**	expires in 30 days

Least privilege **Scoped** **Expiring** **Revocable** **Visible**

Each one shows up separately in sessions, monitoring, audit and permission reviews.

PASS Data Community Summit

What a per-entity identity looks like

```
-- One login per MCP server: read-only, scoped
CREATE LOGIN mcp_reporting_ro
    WITH PASSWORD = N'<from your vault>', CHECK_POLICY = ON;

USE SalesReporting;
CREATE USER mcp_reporting_ro FOR LOGIN mcp_reporting_ro;
GRANT SELECT ON SCHEMA::reporting TO mcp_reporting_ro;
DENY  SELECT ON SCHEMA::pii       TO mcp_reporting_ro;

-- Tag the connection so it is visible everywhere
-- Server=sql-rpt;Database=SalesReporting;
-- ApplicationIntent=ReadOnly;
-- Application Name=mcp-reporting-ro;
```



A login per entity

Never a person's own login, never a shared service account.



Grant by schema, deny PII

An explicit DENY survives someone adding a role later.



Tag every connection

Application Name shows in sessions, monitoring and audit. ReadOnly intent routes to a secondary.

Questions to ask of any MCP server

Ask before you allow it	Example: Redgate Monitor's MCP server
Can it change anything?	Read-only tools; it cannot change Monitor or your databases
Whose identity does it use?	A dedicated token with the MCP role; other roles are rejected
What can it reach?	Token scoped to the servers you choose
When does access end?	Token expiry; revoke by deleting the token
Is it rate limited?	60 calls per minute per token by default
Is it logged?	Every request is logged; HTTPS only

02

A practical scenario

Where access quietly grows beyond what anyone intended

PASS Data Community Summit

One week, five reasonable decisions

Mon	Tue	Wed	Thu	Fri
Chase a bug in staging	Point it at production “just to check”	Share the config with an analyst	Turn on write tools to fix the data	Incident: what did it do?

Effective access, by the end of each day



Monday and Tuesday: the connection moves

Mon Chase a bug in staging

What happened

A developer adds a SQL Server MCP server to their IDE assistant, using their own login.

What quietly grew

The assistant inherits db_owner on staging, and staging holds an unmasked copy of production.

Good practice

A dedicated read-only login for the MCP server, and masked data outside production.

Tue Point it at production

What happened

The bug only reproduces in production, so the connection string changes “just for today”.

What quietly grew

The same personal login, now with production rights, querying the primary in business hours.

Good practice

A separate production identity, a readable secondary, and time-boxed approval.

Wednesday and Thursday: access spreads

Wed Share the config

What happened

An analyst wants the same setup. The MCP config, password included, goes into the team repo.

What quietly grew

Two people and two assistants behind one credential, and the secret lives in Git history.

Good practice

One identity per person and per assistant, secrets from a vault, rotate on exposure.

Thu Turn on write tools

What happened

The assistant proposes a data fix. Write tools and auto-approve get switched on.

What quietly grew

UPDATE and DDL through an identity nobody reviews, and prompt injection now has consequences.

Good practice

Read-only by default. Changes go through your normal reviewed deployment pipeline.

“What did the AI assistant actually do?”

Friday, 4:45 pm. The question most teams still can't answer after an incident.

03

Spotting it in the workload you already monitor

Signals, a query you can run today, and a demo

PASS Data Community Summit

Signals that an assistant is connected



New login or program name

program_name like node, python or mcp-*, or a login you've never seen.



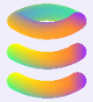
Schema crawling

Bursts of sys.objects, sys.columns and INFORMATION_SCHEMA queries.



Machine-speed bursts

Dozens of distinct ad hoc statements a minute from one session.



Broad reads

SELECT *, TOP 1000 and full scans across many tables in top queries.



Off-pattern timing

Activity from a person's login outside their normal hours.



Permission changes

New role memberships or grants, especially db_owner and sysadmin.

Find it yourself, whatever tools you use

```sql
-- Who is connected, and as what?
SELECT login_name, program_name, host_name,
       COUNT(*) AS sessions,
       MAX(last_request_start_time)
         AS last_active
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
GROUP BY login_name, program_name,
         host_name
ORDER BY sessions DESC;
```

```sql
-- One number to trend and alert on
SELECT COUNT(*)
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
  AND (program_name LIKE N'%mcp%'
       OR login_name LIKE N'mcp[_]%'
       OR login_name LIKE N'agent[_]%');
```

In Redgate Monitor, the second query becomes a custom metric you can chart and alert on.

04

A hardened baseline that shows drift

CIS Benchmarks and configuration compliance

CIS Benchmarks limit the blast radius

Reduce attack surface

xp_cmdshell, CLR and OLE Automation off: a steered prompt can't reach the OS.

Logins and authentication

sa disabled, password policy enforced: no god-mode account to borrow.

Role permissions

Minimal public and guest rights: a new login starts with nothing.

Auditing

Login auditing and an audit in place: there's a record to work from.

Network and instance exposure

Fewer endpoints and remote options: fewer paths for a stray client.

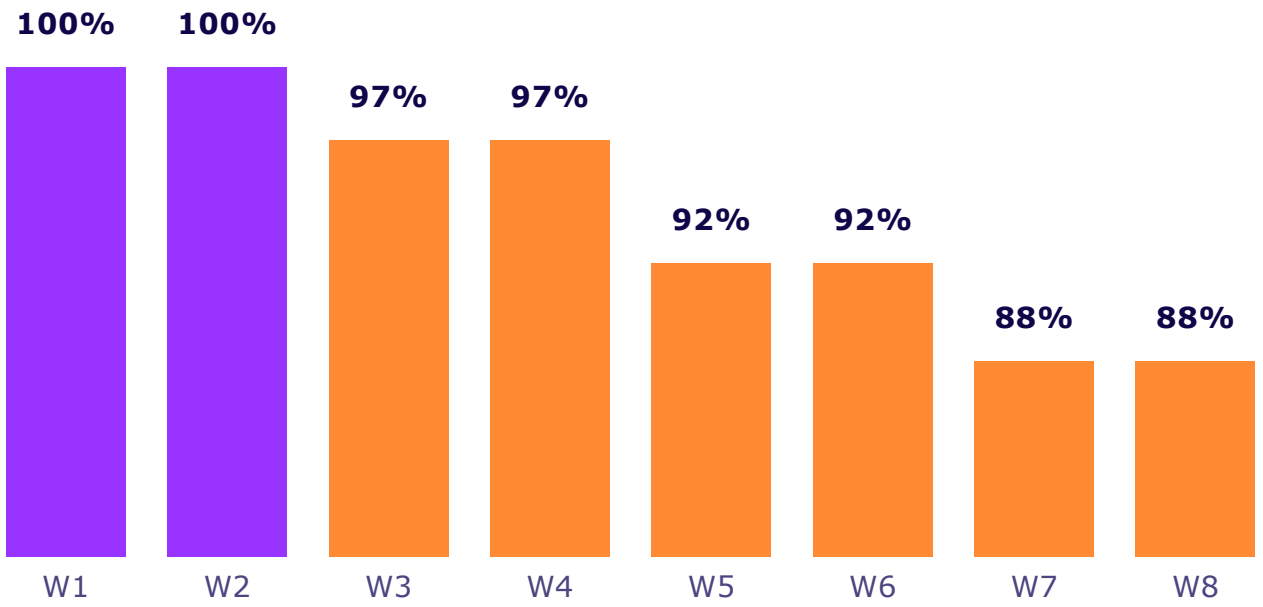
Encryption and backups

Encryption in transit and at rest: results stay protected end to end.

Redgate Monitor includes CIS templates for SQL Server 2019, 2022 and 2025, at server and database level.

Baselines drift one “temporary” fix at a time

Estate compliance with the CIS template, by week



W3 xp_cmdshell enabled for an agent's file export

W5 MCP login added to sysadmin “temporarily”

W7 TRUSTWORTHY ON for a helper database

Make drift visible

- Apply the server and database templates to every instance
- Alert on permission changes, not just failures
- Review non-compliant properties every week
- Treat “temporary” as a ticket with an end date

Demo: CIS baseline and drift in Redgate Monitor

DEMO

01 Apply the CIS SQL Server 2022 server and database templates

02 Estate compliance, then drill into non-compliant properties

03 Introduce drift: enable xp_cmdshell, add the MCP login to sysadmin

04 See it flagged, and the permission-change alert fire

05 Duplicate a template to add checks of your own

Good to know

Configuration compliance and Permissions need Monitor Enterprise, for SQL Server.

Built-in templates are read-only: duplicate one to customize it.

Apply both the server and database template for full benchmark coverage.

05

“What did the AI assistant actually do?”

A practical starting point for answering it

PASS Data Community Summit

Five sources, joined on identity and time

01

Identity

Which login and application? Only possible with one identity per entity.

02

Permissions history

What could it do at that moment?

03

Workload history

Sessions, top queries, waits and blocking for the window.

04

Audit

Logins, permission changes, and reads and writes on sensitive objects.

05

Client and MCP logs

Prompts, tool calls and results, from the assistant's side.

The timeline you can now build

09:02	09:03	09:05	09:11
mcp_reporting_ro logs in from mcp-reporting-ro	47 catalog queries in 40 seconds	SELECT on pii.Customers denied	UPDATE sales.Orders, 1,204 rows

A starter audit for AI identities

```
CREATE SERVER AUDIT AI_Access
    TO FILE (FILEPATH = N'D:\Audit\AI\', MAXSIZE = 256 MB)
    WHERE server_principal_name = N'mcp_reporting_ro';

CREATE SERVER AUDIT SPECIFICATION AI_Access_Server
    FOR SERVER AUDIT AI_Access
    ADD (SUCCESSFUL_LOGIN_GROUP), ADD (FAILED_LOGIN_GROUP),
    ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP)
    WITH (STATE = ON);

USE SalesReporting;
CREATE DATABASE AUDIT SPECIFICATION AI_Access_Db
    FOR SERVER AUDIT AI_Access
    ADD (SELECT, INSERT, UPDATE, DELETE, EXECUTE
         ON DATABASE::SalesReporting BY public),
    ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP)
    WITH (STATE = ON);

ALTER SERVER AUDIT AI_Access WITH (STATE = ON);
```



Filter on the identity

The predicate keeps volume low: only the AI login is recorded.



Server: logins and roles

Who connected, who failed, and every role change.



Database: reads and writes

Statements against the database, including denied ones.



Read it back

sys.fn_get_audit_file, or ship the files to your SIEM.

Start on Monday, whatever tools you use

01 **Inventory what connects**

Group sessions by login, program and host for a week.

02 **Give every entity its own identity**

Scoped, expiring, and tagged with Application Name.

03 **Read-only by default**

Readable secondaries for production questions; writes go through review.

04 **Keep secrets out of config files**

Use a vault or managed identity, and rotate on exposure.

05 **Baseline, and watch for drift**

CIS Benchmark, plus alerts on permission changes.

06 **Make "what did it do?" answerable**

Audit and workload history, kept for your investigation window.

Takeaways

01



Assume it's already connected

Look in the workload you already monitor. It's usually there.

02



One identity per entity

Least privilege, scoped, expiring, and visible everywhere.

03



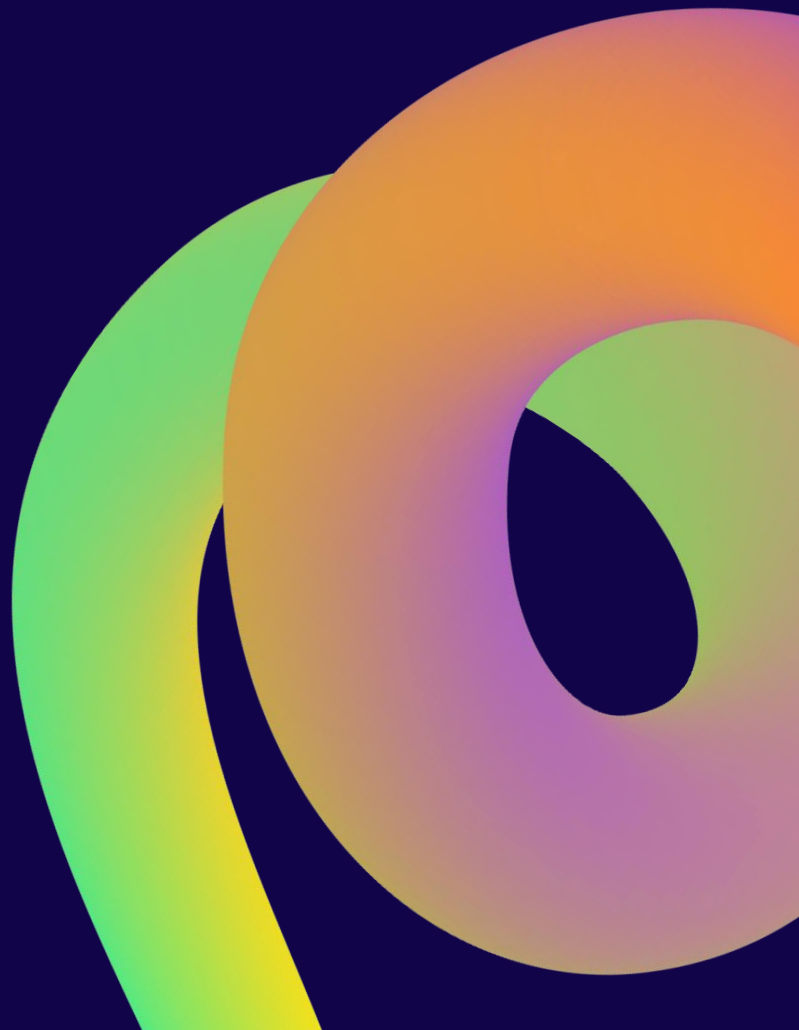
Answer “what did it do?” before you're asked

Hardened baseline, audit, and history, joined on identity and time.

Questions

PASS Data Community Summit

Host redgate



Your feedback is important to us



Evaluate this session at:

passdatacommunitysummit.com/evaluations

Thank you

Questions? Start with one identity, one query and one audit, then build from there.

Kellyn Gorman

@DBAKevlar

dbakevlar.com

red-gate.com/products/redgate-monitor

John Martin

/in/johnqmartin

